



SIMATS ENGINEERING



Saveetha Institute of Medical and Technical Sciences

Chennai 2026

ASSESSMENT TOOL -2

C05_AT2

Name : N.Sunil Kumar

Register No : 192472330

Course code : CSA0708

Course name: Computer Networks

Faculty name: Dr.S.Saraswathi

PSEUDOCODE WRITING TASK

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

ANSWER :-

Mathematical Calculation

- **Throughput:**

$$\text{Throughput} = \frac{\text{Total Data Received (bits)}}{\text{Time (seconds)}} = \frac{8000 \text{ bits}}{4 \text{ s}} = 2000 \text{ bps}$$

- **Packet Loss Percentage:**

$$\begin{aligned} \text{Packet Loss \%} &= \left(\frac{\text{Packets Sent} - \text{Packets Received}}{\text{Packets Sent}} \right) \times 100 = \left(\frac{200 - 180}{200} \right) \times 100 \\ &= 10.0\% \end{aligned}$$

- **Efficiency Classification:**

- Throughput condition (> 1500 bps): Met ($2000 > 1500$)
- Packet Loss condition ($< 10\%$): Not Met ($10\% \nlessgtr 10\%$)
- Classification: Inefficient

Pseudocode Algorithm

```
BEGIN NetworkPerformanceAnalyzer
```

```
// 1. Initialize input variables
```

```
SET totalBitsReceived = 8000
```

```

SET timeSeconds = 4.0

SET packetsSent = 200

SET packetsReceived = 180

// 2. Input validation to prevent division by zero

IF timeSeconds <= 0 OR packetsSent <= 0 THEN

    PRINT "Error: Time duration and total packets sent must be greater than zero."

    EXIT

ENDIF

IF packetsReceived > packetsSent THEN

    PRINT "Error: Packets received cannot exceed packets sent."

    EXIT

ENDIF

// 3. Perform calculations

SET throughput = totalBitsReceived / timeSeconds

SET packetsLost = packetsSent - packetsReceived

SET packetLossPercentage = (packetsLost / packetsSent) * 100.0

// 4. Determine performance classification

IF throughput > 1500 AND packetLossPercentage < 10 THEN

    SET networkStatus = "Efficient"

ELSE

    SET networkStatus = "Inefficient"

ENDIF

// 5. Output complete summary report

PRINT "=====
```

```

PRINT "    NETWORK PERFORMANCE REPORT    "

PRINT "===== "

PRINT "Data Received    : " + totalBitsReceived + " bits"

PRINT "Duration        : " + timeSeconds + " seconds"

PRINT "Packets Sent      : " + packetsSent

PRINT "Packets Received  : " + packetsReceived

PRINT "Packets Lost      : " + packetsLost

PRINT "-----"

PRINT "Throughput        : " + throughput + " bps"

PRINT "Packet Loss       : " + packetLossPercentage + " %"

PRINT "Classification    : " + networkStatus

PRINT "===== "

END

```

2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

ANSWER :-

Pseudocode Algorithm: Bandwidth Utilization Monitoring

ALGORITHM Bandwidth_Monitor

BEGIN

 // Number of network links

 INPUT numberOfLinks

```
// Create array to store bandwidth usage

DECLARE bandwidthUsage[numberOfLinks]

WHILE TRUE DO

    DISPLAY "----- Bandwidth Monitoring -----"

    // Monitor each network link

    FOR i ← 1 TO numberOfLinks DO

        INPUT bandwidthUsage[i]

        DISPLAY "Link ", i, " Utilization: ", bandwidthUsage[i], "%"

        // Check for overloaded links

        IF bandwidthUsage[i] > 80 THEN

            DISPLAY "Warning: Link ", i, " is overloaded."

            DISPLAY "Recommendation: Switch traffic to the backup link."

        ELSE

            DISPLAY "Link ", i, " is operating normally."

        ENDIF

    ENDFOR

    DISPLAY "Monitoring cycle completed."

    // Wait before the next monitoring cycle

    WAIT 5 seconds

ENDWHILE

END
```

Explanation

This algorithm supports efficient bandwidth management in the following ways:

1. **Continuous Monitoring:** It continuously checks the bandwidth utilization of all network links using a loop, ensuring real-time network monitoring.
2. **Data Storage:** The utilization values are stored in an array (`bandwidthUsage`), making it easy to manage and analyze the bandwidth usage of multiple links.
3. **Overload Detection:** It compares each link's bandwidth usage with the threshold of 80%. If the utilization exceeds this limit, the algorithm identifies the link as overloaded.
4. **Traffic Redirection:** When an overloaded link is detected, it recommends switching traffic to a backup link, reducing the load on the busy link.
5. **Congestion Prevention:** By detecting high utilization early and suggesting traffic redistribution, the algorithm helps prevent network congestion, improves network performance, and ensures reliable communication between branches.
6. **Scalability:** The algorithm works for any number of network links by simply changing the value of `numberOfLinks`, making it suitable for both small and large networks.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

ANSWER :-

Link Quality Scoring Model

To balance capacity, speed, and congestion, a normalized composite score is calculated:

$$\text{Score} = \left(w_1 \times \frac{\text{Bandwidth}}{\text{Max Bandwidth}} \right) + \left(w_2 \times \left(1 - \frac{\text{Delay}}{\text{Max Delay}} \right) \right) + \left(w_3 \times \left(1 - \frac{\text{Utilization}}{100} \right) \right)$$

- $w_1 = 0.40$ (Bandwidth), $w_2 = 0.35$ (Low Delay), $w_3 = 0.25$ (Available Capacity)
- Higher score indicates higher link quality (range: 0.0 to 1.0).

Pseudocode Algorithm

BEGIN DynamicPathOptimizer

 // 1. Network Topology Setup

 SET branchList = ["Branch_1", "Branch_2", "Branch_3", "Branch_4", "Branch_5", "Branch_6"]

 SET MAX_BANDWIDTH = 1000.0 // 1000 Mbps normalization baseline

 SET MAX_DELAY = 100.0 // 100 ms normalization baseline

 SET UTILIZATION_LIMIT = 85.0 // Overload threshold (%)

 SET POLLING_INTERVAL = 15 // Sampling frequency in seconds

 SET isRunning = TRUE

 // 2. Continuous Monitoring Loop

 WHILE isRunning DO

 PRINT "===== Dynamic Route Optimization Cycle ====="

 FOR EACH branch IN branchList DO

 SET candidatePaths = GET_PATHS_TO_BRANCH("HQ", branch)

 SET bestPath = NULL

 SET highestScore = -1.0

```

FOR EACH path IN candidatePaths DO

    // Collect real-time telemetry metrics

    SET isUp = GET_LINK_STATUS(path)

    SET bw = GET_BANDWIDTH_MBPS(path)

    SET delay = GET_DELAY_MS(path)

    SET util = GET_UTILIZATION_PERCENT(path)

    // Check for failure or link overload

    IF isUp == FALSE THEN

        PRINT "[CRITICAL] Path " + path + " to " + branch + " is DOWN."

        TRIGGER_ALERT("LINK_FAILURE", path, branch)

        CONTINUE // Skip down link

    ELSE IF util > UTILIZATION_LIMIT THEN

        PRINT "[WARNING] Path " + path + " to " + branch + " OVERLOADED

            (" + util + "%)."

        TRIGGER_ALERT("LINK_OVERLOAD", path, branch)

        CONTINUE // Skip degraded link to reroute dynamically

    ENDIF

    // Calculate Normalized Link Quality Score

    SET normBW = MIN(bw / MAX_BANDWIDTH, 1.0)

    SET normDelay = 1.0 - MIN(delay / MAX_DELAY, 1.0)

    SET normAvail = 1.0 - (util / 100.0)

    SET pathScore = (0.40 * normBW) + (0.35 * normDelay) + (0.25 * normAvail)

    // Track optimal path

    IF pathScore > highestScore THEN

```



```

        SET highestScore = pathScore

        SET bestPath = path

    ENDIF

ENDFOR

// Route Enforcement

IF bestPath != NULL THEN

    APPLY_ROUTING_TABLE_UPDATE("HQ", branch, bestPath)

    PRINT "[OPTIMAL] " + branch + " -> " + bestPath + " (Score: " +
ROUND(highestScore, 2) + ")"

ELSE

    PRINT "[ALERT] No healthy path found for " + branch + "! Immediate intervention
required."

    TRIGGER_ALERT("NO_VIABLE_PATH", branch)

ENDIF

ENDFOR

PRINT

WAIT(POLLING_INTERVAL)

ENDWHILE

END

```

System Architecture Benefits

- **Routing Efficiency:** Moves beyond standard static metrics (like hop-count) by scoring candidate paths using composite real-time parameters (bandwidth capacity, latency, and live utilization).
- **Fault Tolerance:** Failed links (`isUp == FALSE`) are dropped immediately from consideration in the current cycle, triggering failover routes within the sampling interval without traffic blackholing.

- Network Reliability & Congestion Control: Filtering out saturated links ($> 85\%$) prevents cascading packet drops, excessive queueing delays, and bufferbloat before user applications degrade.
- Dynamic Adaptation: The periodic sampling loop continually re-evaluates fluctuating load throughout business hours, maintaining balanced traffic across all six branches.